KLASTROWANIE SEMANTYCZNE · WARSTWA ZNACZENIA

PIĘĆ REDAKCJI. JEDNA HISTORIA.

Embedding to wektor, czyli lista liczb o stałej długości, którą model językowy oblicza dla danego fragmentu tekstu. Teksty o podobnym znaczeniu zwykle otrzymują wektory położone blisko siebie, więc system może obliczyć ich podobieństwo, zamiast oceniać je na wyczucie.

Kiedy na przykład prezydent rezygnuje z urzędu, pięć redakcji opisuje to wydarzenie na pięć różnych sposobów. Teksty mogą nie mieć wspólnych sformułowań, a czasem są napisane w różnych językach. Program, który grupuje artykuły na podstawie wspólnych słów, nie rozpozna nawet prostej parafrazy.

System przedstawia każdy artykuł jako punkt w przestrzeni znaczeń. Nie mierzy zwykłej odległości między punktami. Zamiast tego porównuje kierunki prowadzących do nich wektorów za pomocą cosinusa kąta. Wynik pozwala oszacować, czy teksty są ze sobą powiązane.

768
LICZB NA ARTYKUŁ
Tyle współrzędnych opisuje położenie każdego tekstu

0,70
PRÓG PODOBIEŃSTWA
Podobieństwo cosinusowe mieści się w przedziale od −1,00 do 1,00. Przy wyniku co najmniej 0,70 dwa teksty trafiają do jednej grupy; w pokazanej próbce ta sama historia osiąga wynik 0,9933, a inna 0,3872

3
TEKSTY NA WYWOŁANIE
Serwer może mieć 4 GB pamięci i nie korzystać z pamięci wymiany (swapu), dlatego teksty trafiają do modelu małymi porcjami

brak
TERMINU WAŻNOŚCI
Raz obliczony wynik się nie zmienia, więc nie trzeba go unieważniać

ELEMENTARZ

ZNACZENIE ZAPISANE JAKO WSPÓŁRZĘDNE

Trzy właściwości sprawiają, że embeddingi nadają się do tego zadania.

01 TEKST JAKO 768 WSPÓŁRZĘDNYCH

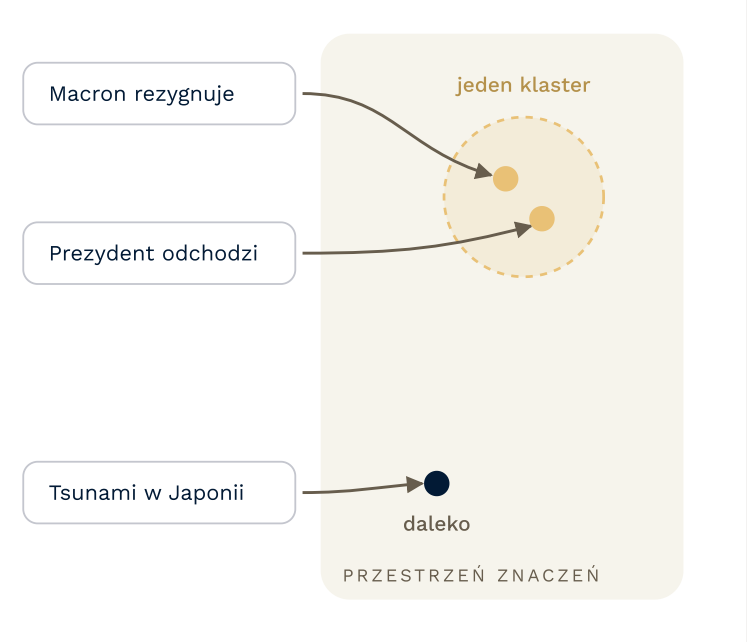
Model zamienia zdanie w listę 768 liczb. Wynik nie jest streszczeniem ani zestawem słów kluczowych. Te liczby są współrzędnymi, które wyznaczają położenie tekstu w przestrzeni znaczeń. Każdy tekst trafia do tej samej przestrzeni.

02 BLISKOŚĆ WSKAZUJE NA POKREWIEŃSTWO

Teksty o tym samym wydarzeniu często trafiają prawie w to samo miejsce, mimo że używają innych słów. To „prawie” da się zmierzyć. Służy do tego miara oparta na kącie między wektorami: 1,00 oznacza ten sam kierunek, 0,00 kierunki prostopadłe, a −1,00 kierunki przeciwne. Dzięki temu nie trzeba ręcznie pisać reguł dotyczących synonimów i szyku zdania.

03 TEN SAM TEKST ZAWSZE DAJE TE SAME LICZBY

Obliczenie embeddingu jest deterministyczne: ten sam tekst przetworzony przez tę samą wersję modelu zawsze daje dokładnie ten sam wektor. Wynik nie zależy od chwili obliczenia ani od tekstów wysłanych wcześniej. Dlatego warto go zapisać i nie obliczać ponownie.

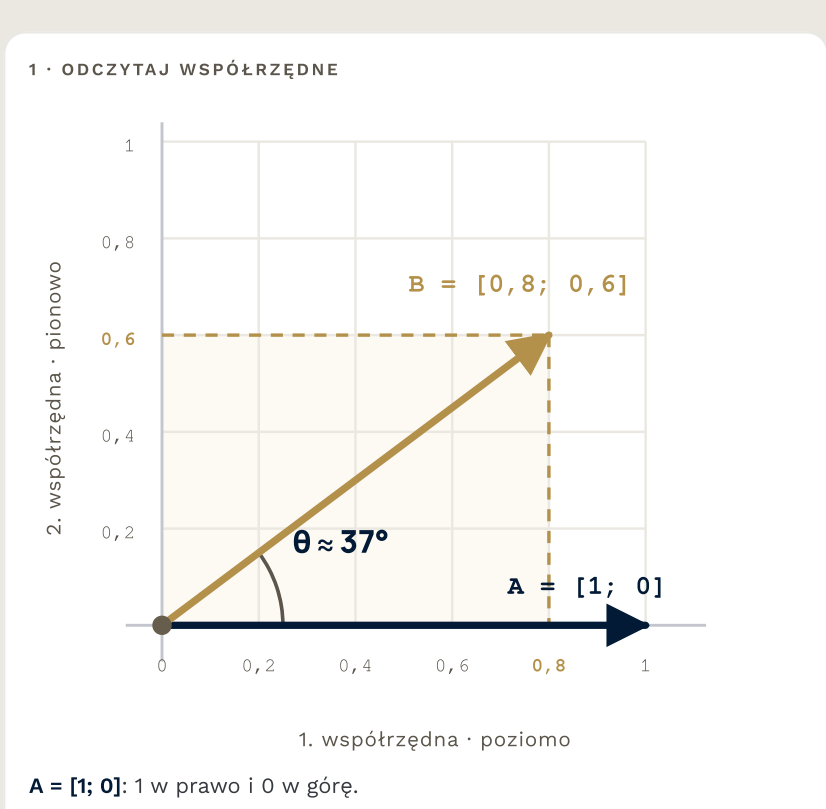


inne słowa · punkty blisko siebie

OD STRONY TECHNICZNEJ

Wektor to lista liczb. W tym przykładzie używamy dwóch współrzędnych, żeby łatwo pokazać rysunek i rachunek. Wektory A i B dobraliśmy do przykładu; nie pochodzą z modelu. Obie strzałki zaczynają się w punkcie [0; 0]. Pierwsza współrzędna mówi, jak daleko przesunąć koniec strzałki w poziomie, a druga w pionie. Nie trzeba mierzyć kąta kątomierzem. Jego cosinus można policzyć bezpośrednio ze współrzędnych obu strzałek.

1 · ODCZYTAJ WSPÓŁRZĘDNE



A = [1; 0]: 1 w prawo i 0 w górę.
B = [0,8; 0,6]: 0,8 w prawo i 0,6 w górę.

2 · POLICZ PODOBIEŃSTWO KIERUNKÓW

$$\cos(\theta) = \frac{A \cdot B}{|A| \times |B|}$$

$A \cdot B$ to iloczyn skalarny: pomnóż pary liczb i dodaj wyniki.

$|A|$ i $|B|$ to długości strzałek. Obie muszą być większe od zera, bo nie można dzielić przez zero.

1. Policz licznik. Pomnóż liczby na tych samych pozycjach i dodaj:
   $(1 \times 0{,}8) + (0 \times 0{,}6) = 0{,}8$
2. Oblicz długości wektorów z twierdzenia Pitagorasa: podnieś współrzędne do kwadratu, dodaj wyniki i wyciągnij pierwiastek:
   $|A| = \sqrt{1^2 + 0^2} = 1$
   $|B| = \sqrt{0{,}8^2 + 0{,}6^2} = \sqrt{0{,}64 + 0{,}36} = 1$
3. Podziel licznik przez iloczyn długości. Dzielenie usuwa wpływ rozmiaru strzałek, więc zostaje porównanie ich kierunków:
   $0{,}8 \div (1 \times 1) = 0{,}80$

0,80 Wynik przekracza przyjęty próg 0,70. Gdyby były to wektory dwóch tekstów, system połączyłby je w jedną grupę. Wynik 0,80 nie oznacza 80% pewności, że teksty opisują to samo wydarzenie.

−1 · przeciwne 0 · prostopadłe 1 · ten sam kierunek

Na rysunku widać kąt około 37°, ponieważ $\arccos(0{,}80) \approx 37°$. System nie potrzebuje samego kąta. Do porównania tekstów używa bezpośrednio wyniku 0,80.

W używanym tu modelu wektor ma 768 współrzędnych. Aby obliczyć iloczyn skalarny, program mnoży 768 par liczb i dodaje wyniki. Ollama przez `/api/embed` zwraca wektory znormalizowane do długości 1 (normalizacja L2), z możliwymi drobnymi odchyleniami wynikającymi z precyzji obliczeń. Dla wektorów o długości 1 mianownik wynosi 1, więc cosinus jest równy iloczynowi skalarnemu. Pełny wzór działa również dla wektorów o innych, niezerowych długościach.

PUNKT ODNIESIENIA

DLACZEGO NIE WYSTARCZY LICZENIE SŁÓW

Zanim upowszechniły się embeddingi, do porównywania tekstów często stosowano metodę **TF-IDF**. Najpierw liczy ona wystąpienia słów, a następnie przypisuje każdemu z nich wagę zależną od częstości w danym tekście i rzadkości w całym zbiorze. Dwa teksty uznaje za podobne, gdy występują w nich te same słowa o wysokich wagach.

JAK TF-IDF PORÓWNUJE TEKSTY

O wadze każdego słowa decydują dwie wielkości: częstość występowania w tym tekście oraz rzadkość w całym zbiorze. Słowo obecne wszędzie, takie jak „oraz” albo „jest”, nie odróżnia jednego tekstu od drugiego i otrzymuje wagę bliską zeru. Słowo rzadkie, takie jak „rezygnacja”, otrzymuje dużą wagę.

Tekst zamienia się w listę takich wag, a podobieństwo dwóch tekstów jest miarą tego, jak bardzo obie listy się pokrywają. Nie potrzeba tu modelu językowego ani uczenia maszynowego. Całość sprowadza się do obliczeń opartych na częstości słów, które można zamknąć w kilkudziesięciu wierszach kodu i uruchomić bez żadnych przygotowań.

KIEDY TF-IDF NIE ROZPOZNAJE PODOBIEŃSTWA

Problem pojawia się, gdy dwa teksty mówią to samo innymi słowami. „Prezydent ustępuje” i „Macron rezygnuje” nie mają ani jednego wspólnego, istotnego słowa, więc ich podobieństwo wynosi dokładnie zero.

Żeby metoda rozpoznała podobieństwo tych zdań, trzeba dopisać listę synonimów, reguły sprowadzania słów do form podstawowych i wykaz nazwisk. Ktoś musi przygotować je ręcznie, a potem powtórzyć tę pracę dla każdego języka z osobna.

NAJTRUDNIEJSZY PRZYPADEK: KILKA JĘZYKÓW NARAZ

To samo wydarzenie może być jednocześnie opisywane po angielsku, francusku i polsku. Jeśli artykuły w dwóch językach nie mają wspólnych słów, metoda oparta na ich liczeniu nada im podobieństwo równe zeru, nawet gdy opisują to samo wydarzenie. Bez dodatkowych reguł trzeba by najpierw przetłumaczyć wszystko na jeden język. Wymagałoby to kolejnego modelu, a jego błędy wpływałyby na wyniki.

W języku polskim dochodzi jeszcze odmiana: „rezygnacja”, „rezygnacji” i „rezygnacją” to dla licznika trzy różne słowa. Model wielojęzyczny może porównywać teksty bez osobnych reguł dla każdego języka. Ponieważ uczył się wielu języków naraz, umieszcza napisane w nich teksty we wspólnej przestrzeni. Polskie zdanie może więc znaleźć się blisko swojego angielskiego odpowiednika.

PYTANIE	TF-IDF	EMBEDDINGI
Co porównuje?	wspólne słowa	sens całego tekstu
Skąd wie, co jest ważne?	z tego, jak rzadkie są słowa w bieżącym zbiorze tekstów	z modelu, który wcześniej przeczytał ogromną liczbę tekstów
Rozpozna inne sformułowanie tej samej rzeczy?	nie, dopóki ktoś nie dopisze słownika synonimów	zwykle tak, ponieważ model został wytrenowany do porównywania znaczenia
Poradzi sobie z dwoma językami?	nie bez dodatkowej obsługi, na przykład tłumaczenia	tak, jeśli model jest wielojęzyczny
A z nazwiskiem, symbolem giełdowym, numerem ustawy?	lepiej: dokładne trafienie rzadkiego słowa waży najwięcej	gorzej: podobne nazwy potrafią wylądować blisko siebie
Czy wynik jest zawsze taki sam?	nie, zmienia się razem ze zbiorem, w którym liczona jest rzadkość słów	tak, ten sam tekst i model zawsze dają ten sam wynik
Ile kosztuje?	niewiele; obliczenia odbywają się lokalnie, bez dodatkowych narzędzi	model zajmuje 1,3 GB, a obliczenie trwa kilka sekund; wynik zapisuje się raz na zawsze
Da się sprawdzić, dlaczego tak wyszło?	tak, widać, które słowa zadecydowały	nie; to 768 liczb, z których żadna z osobna nic nie znaczy

Żadna z tych metod nie jest bezwzględnie lepsza. Wyszukiwarki zwykle łączą oba podejścia: liczenie słów dobrze rozpoznaje numer ustawy, a embeddingi oddają sens pytania. Tutaj trzeba ustalić, czy artykuły opisują to samo wydarzenie, więc liczy się przede wszystkim ich sens.

DLACZEGO ZOSTAŁA TYLKO JEDNA METODA

Początkowo sposób grupowania zależał od ustawienia: środowisko programisty bez modelu korzystało z TF-IDF, a środowisko produkcyjne z embeddingów. W rezultacie ta sama porcja artykułów układała się w inne grupy w środowisku programisty niż w środowisku produkcyjnym, a jeden próg podobieństwa musiał obsłużyć dwa różne rozkłady wyników. Gdy na podstawie grup artykułów powstają sygnały mogące wspierać decyzje inwestycyjne, „u mnie wyszło inaczej” przestaje być drobiazgiem.

Jedyną przewagą liczenia słów był brak zależności od modelu. Zniknęła ona po jednorazowym obliczeniu embeddingów dla zbioru danych używanego w testach i zapisaniu ich w pliku danych testowych dołączonym do kodu. Przed każdym scenariuszem testowym gotowe wektory są wczytywane do pamięci podręcznej. Testy korzystają więc z tych samych embeddingów i tej samej metody grupowania co środowisko produkcyjne, ale nie muszą wywoływać modelu.

MODEL I INFRASTRUKTURA

SKĄD BIORĄ SIĘ TE LICZBY I GDZIE DZIAŁA MODEL?

Model musiał obsługiwać wiele języków, zmieścić się na jednym serwerze z bazą danych i aplikacją oraz działać we własnej infrastrukturze, bez wysyłania treści na zewnątrz.

NOMIC-EMBED-TEXT-V2-MOE

To otwarty model, który zamienia tekst na 768 liczb. Nie prowadzi rozmowy, nie streszcza tekstów ani nie odpowiada na pytania.

Skrót **MoE** w nazwie to *mixture of experts*: model zawiera kilka wyspecjalizowanych podsieci, ale dla każdego fragmentu tekstu uruchamia tylko część z nich. Ma więc możliwości większego modelu, choć zużywa podobną ilość zasobów co mniejszy model.

W tym zadaniu liczy się przede wszystkim wielojęzyczność: francuska depesza i jej angielskie omówienie mogą trafić blisko siebie w przestrzeni, bez tłumaczenia i bez ręcznie napisanych reguł.

DANE TECHNICZNE

ROZMIAR I KSZTAŁT

WYJŚCIE	768 liczb · da się skrócić kosztem jakości
JĘZYKI	wielojęzyczny · około stu języków
ILE CZYTA NARAZ	~512 tokenów (~2 akapity) · dalsza część jest obcinana
WAGI	otwarte · pobierane raz, trzymane lokalnie
W PAMIĘCI	~1,3 GB · tylko na czas liczenia porcji
GDZIE DZIAŁA	Ollama na tym samym serwerze · bez połączeń z siecią zewnętrzną

Model czyta około 512 tokenów, więc dalsza część tekstu jest obcinana. W tym systemie przyjęto, że temat artykułu powinien być widoczny w nagłówku i pierwszym akapicie. Jeśli istotna informacja pojawi się później, nie wpłynie na wektor.

? CZYM JEST OLLAMA

To program, który uruchamia modele językowe na własnym komputerze albo serwerze. Ollamę instaluje się raz, a model pobiera się jednym poleceniem. Działa wtedy pod lokalnym adresem, podobnie jak baza danych, do której aplikacja wysyła zapytanie i z której otrzymuje wynik.

Ollama otrzymuje listę tekstów i zwraca listę wektorów. Aplikacja wywołuje ją przez HTTP tylko wtedy, gdy trzeba obliczyć nowy wynik. Żaden tekst nie wychodzi przy tym poza serwer, a za lokalne wywołanie nie płaci się osobno.

ZŁ CZEGO WYMAGA URUCHOMIENIE

Model tej wielkości zajmuje w pamięci ~1,3 GB, wyłącznie na czas liczenia porcji tekstów. Mieści się obok bazy danych i samej aplikacji na zwykłym VPS-ie za kilkadziesiąt złotych miesięcznie. Nie wymaga karty graficznej ani konta u dostawcy chmury.

Serwer może mieć 4 GB pamięci i nie korzystać z pamięci wymiany (swapu). W takiej konfiguracji model jest usuwany z pamięci zaraz po policzeniu porcji, a teksty trafiają do niego po trzy. Ponowne załadowanie trwa kilka sekund i jest potrzebne tylko wtedy, gdy porcja zawiera nową treść.

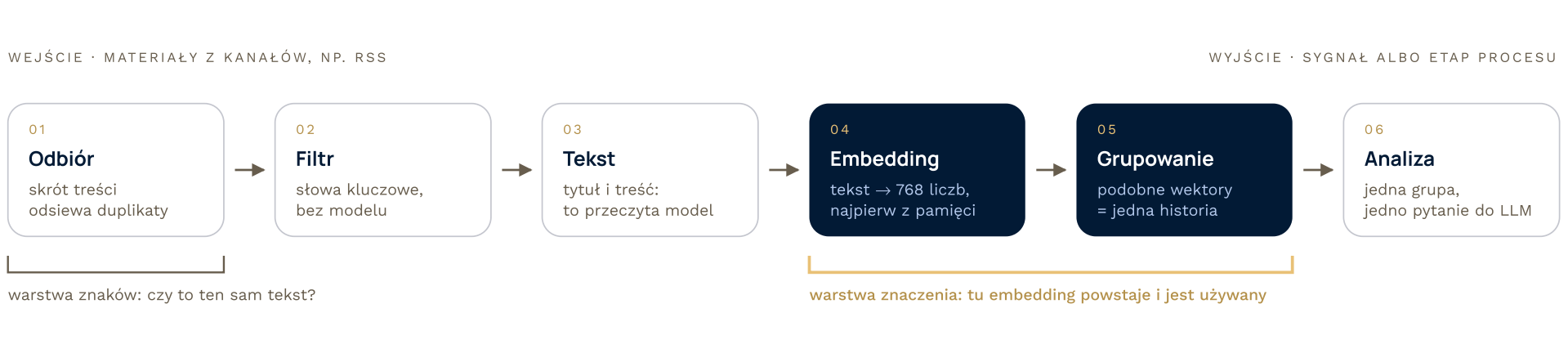
OD STRONY TECHNICZNEJ

`POST /api/embed`, wejście w paczkach po 3, `keep_alive: 0` przy ostatniej paczce (zwolnienie pamięci). Adres i nazwa modelu pochodzą z konfiguracji (`OLLAMA_EMBED_MODEL`).

OBIEG

JAKĄ ROLĘ W TYM PROCESIE ODGRYWA EMBEDDING

Artykuł przechodzi sześć kroków od kanału informacyjnego do sygnału. Tylko dwa wymagają embeddingu. Pozostałe ograniczają liczbę wywołań modelu i pomagają uzyskać wiarygodny wynik.


WEJŚCIE · MATERIAŁY Z KANAŁÓW, NP. RSS
WYJŚCIE · SYGNAŁ ALBO ETAP PROCESU
01 Odbiór skrót treści odsiewa duplikaty
02 Filtr słowa kluczowe, bez modelu
03 Tekst tytuł i treść: to przeczyta model
04 Embedding tekst → 768 liczb, najpierw z pamięci
05 Grupowanie podobne wektory = jedna historia
06 Analiza jedna grupa, jedno pytanie do LLM
warstwa znaków: czy to ten sam tekst?
warstwa znaczenia: tu embedding powstaje i jest używany

Kroki od 01 do 03 nie analizują znaczenia: pierwszy porównuje znaki, drugi szuka słów z listy, a trzeci składa tekst dla modelu. Analiza znaczenia zaczyna się w kroku 04. Jej wynik zostaje zapisany jako wiersz liczb i wykorzystany w kroku 05. Krok 06 otrzymuje gotową grupę artykułów, nie wektor.

≡ ODBIÓR (KROK 01): CZY TEN TEKST JUŻ TU BYŁ?

Ta sama depesza może przyjść z dwóch kanałów albo ponownie po przerwaniu połączenia. Program wykrywa duplikat za pomocą **skrótu treści**, czyli krótkiego ciągu znaków obliczonego z całego tekstu. Ten sam tekst zawsze daje ten sam skrót, a każda zmiana daje inny. Jeśli taki skrót jest już w bazie, program nie zapisuje artykułu ponownie.

Rozstrzygnięcie jest zerojedynkowe i zupełnie nie uwzględnia znaczenia: przesunięcie choćby jednego przecinka daje inny skrót i drugi wiersz w bazie.

≈ EMBEDDING I GRUPOWANIE (KROKI 04 I 05): CZY TO TA SAMA HISTORIA?

Pięć redakcji opisuje jedno wydarzenie pięcioma różnymi tekstami, więc skrót każdego z nich będzie inny. Dopiero embedding pozwala zmierzyć, jak blisko siebie leżą te teksty, i połączyć je w jedną grupę, mimo że nie mają ani jednego wspólnego zdania.

Każde z tych pytań wymaga innego narzędzia. Najpierw program porównuje skróty treści. To szybka operacja, która nie wymaga modelu. Dopiero teksty, które przeszły ten etap, są porównywane pod względem znaczenia, co wymaga uruchomienia modelu.

01 PRZYJMIJ, ALE TYLKO RAZ

Artykuły przychodzą z kanałów informacyjnych, czasem więcej niż raz. Duplikat może pojawić się, gdy system automatycznie ponawia wysyłkę po przekroczeniu czasu oczekiwania albo gdy ta sama depesza trafia do dwóch kanałów. W obu przypadkach skrót treści jest taki sam, więc program odrzuca drugi egzemplarz.

OD STRONY TECHNICZNEJ

`WorldEvent` · o tożsamości wydarzenia rozstrzyga identyfikator obliczany z nagłówka i treści za pomocą szybkiej, niekryptograficznej funkcji skrótu (`xxh128(title + content)`). To klucz **deduplikacji**, celowo inny niż ten, którym posługuje się pamięć podręczna embeddingów.

WEJŚCIE · `materiały z RSS` **WYJŚCIE** · `utrwalone wydarzenia`

02 ODRZUĆ TO, CO NA PEWNO JEST NIEISTOTNE

Następnie prosty filtr sprawdza słowa wymagane i dyskwalifikujące oraz, opcjonalnie, czy źródło znajduje się na liście zaufanych. Nie wymaga modelu i ogranicza liczbę tekstów, dla których trzeba go uruchomić.

Jeśli nikt nie skonfigurował słów kluczowych, nie przechodzi nic. Przepuszczenie wszystkich tekstów mogłoby ukryć błąd konfiguracji aż do otrzymania rachunku.

Listę trzeba jednak najpierw przygotować. Duży model językowy może pomóc podczas konfiguracji, ale nie bierze udziału w przetwarzaniu. Operator opisuje, czego szuka, a model przygotowuje warianty, formy pokrewne i odpowiedniki w językach materiałów źródłowych. Operator sprawdza i zatwierdza ostateczną listę.

OD STRONY TECHNICZNEJ

`WorldEventFilter → WorldEventFilterSpec`: bramka oparta na wyrażeniach regularnych, działająca wyłącznie na CPU. Przy pustej konfiguracji nie przepuszcza żadnego tekstu, zamiast domyślnie przepuszczać wszystkie. Podpowiadanie słów to osobna ścieżka, wywoływana z ekranu konfiguracji: `KeywordSuggesterInterface::suggest(opis, języki)`, adapter `AnthropicKeywordSuggester` z pamięcią podręczną i limitem liczby zapytań.

WEJŚCIE · `migawka wydarzenia` **WYJŚCIE** · `array: obiekty WorldEventDescriptor`

03 USTAL, CO PRZECZYTA MODEL

Dla każdego artykułu kod łączy nagłówek z treścią. Model otrzyma właśnie ten tekst; od jego brzmienia zależą zarówno wektor, jak i klucz w pamięci podręcznej.

OD STRONY TECHNICZNEJ

`FilteredWorldEvent::getTextForClustering()` → `title . ' ' . content`.

WYJŚCIE · `array: teksty`

04 ZAMIEŃ TEKST NA LICZBY

Model zużywa więcej zasobów niż pozostałe kroki, dlatego system najpierw sprawdza, czy tekst został już wcześniej przeliczony. Model otrzymuje wyłącznie teksty, dla których nie ma zapisanego wyniku.

WEJŚCIE · `array: teksty` **WYJŚCIE** · `array: wektory po 768 floatów`

OD STRONY TECHNICZNEJ

W PHP wynik ma typ `array`: zawiera wektory, a każdy z nich jest tablicą 768 liczb typu `float`. PHPDoc zapisuje ten kształt dokładniej jako `list<list<float>>`.

05 GRUPUJ WEDŁUG PODOBIEŃSTWA

Program oblicza podobieństwo osobno dla każdej pary artykułów w dziennej porcji. Jeśli wynik pary wynosi co najmniej 0,70, oba artykuły trafiają do tej samej grupy. Grupy łączą się przez wspólny artykuł: gdy pary A z B oraz B z C osiągają próg, A, B i C tworzą jedną grupę, nawet jeśli para A z C go nie osiąga. Dzienna porcja obejmuje tylko kilkadziesiąt artykułów, dlatego można obliczyć wynik każdej pary bez przybliżeń.

OD STRONY TECHNICZNEJ

`SimilarityGrouping`: podobieństwo liczone jest osobno dla każdej pary, więc przy pięćdziesięciu artykułach potrzeba niecałych 1250 porównań po 768 liczb, czyli ułamek sekundy. Pary z wynikiem co najmniej 0,70 scala struktura zwana **union-find**: każdy artykuł zaczyna jako osobna grupa, a scalenie pary łączy grupy, do których należą tworzące ją artykuły. Dlatego połączenie A z B oraz B z C tworzy jedną grupę ABC, nawet jeśli para A z C nie osiąga progu. Algorytm niczego nie szacuje ani nie próbkuje; podział wynika wprost z progu.

WEJŚCIE · `array: wektory po 768 floatów` **WYJŚCIE** · `indeks → grupa`

06 JEDNA GRUPA, JEDNA ANALIZA

Cała grupa przechodzi jedną analizę, więc pięć doniesień o tym samym wydarzeniu wymaga łącznie jednej analizy zamiast pięciu. Zmniejsza to liczbę analiz na ostatnim etapie. Wynikiem jest sygnał transakcyjny albo informacja, że dłuższy proces przeszedł do kolejnego etapu.

OD STRONY TECHNICZNEJ

`WorldEventCluster`: komponent grupujący ma dwóch odbiorców: `SignalGenerationKernel` (wydarzenia → sygnały) i `ProcessStepper`, który sortuje klastry według wielkości i zatrzymuje się na pierwszym, który przejdzie bramkę etapu.

WYJŚCIE · `Signal` **WYJŚCIE** · `obserwacja etapu`

ZAPAMIĘTYWANIE

JAK WYNIKI MODELU TRAFIAJĄ DO BAZY

Pamięć podręczna zapisuje każdy obliczony wektor, aby model nie liczył go ponownie. Jedna klasa zarządza pamięcią i przekazuje modelowi teksty, dla których brakuje wyniku.

NAJPIERW PYTANIE, POTEM LICZENIE

Program oblicza skrót każdego tekstu, podobnie jak przy wykrywaniu duplikatów. Tym razem podstawą jest dokładnie ta treść, którą otrzyma model. Następnie program wysyła skróty całej porcji do bazy w jednym zapytaniu i sprawdza, które już w niej są.

Do modelu trafiają tylko pozostałe teksty. Program zapisuje każdy wynik, zanim go użyje. Gdy ten sam tekst pojawi się ponownie, odczyta gotowy wektor z bazy.

WZORZEC: DEKORATOR

Klasa, która odczytuje dane z bazy, nie oblicza embeddingów. Klasa obliczająca embeddingi nie zna bazy. Obie udostępniają ten sam interfejs, więc pierwsza może opakować drugą. Tak działa wzorzec dekoratora.

Pozostała część programu zleca jedynie zamianę tekstów na wektory i nie musi wiedzieć, czy wynik pochodzi z bazy, czy z modelu. Pamięć podręczną można dzięki temu usunąć, wymienić lub uzupełnić o kolejną warstwę bez zmiany kodu, który z niej korzysta.

PRZYKŁAD DEKORATORA W KODZIE

W tej skróconej wersji pominięto logowanie, sprawdzanie długości odpowiedzi i obsługę powtórzonych tekstów w jednej porcji.

```php
final readonly class CachedTextVectorizer implements TextVectorizerInterface
{
    public function __construct(
        private TextVectorizerInterface $inner,   // model (Ollama)
        private EmbeddingStoreInterface $store,   // tabela embedding_cache
        private string $modelId,
        private bool $ollamaEnabled,
    ) {}

    public function vectorize(array $texts): array
    {
        $hashOf  = array_map(self::keyFor(...), $texts);
        $vectors = $this->store->getMany($hashOf, $this->modelId);   // 1 zapytanie

        $misses = $this->collectMisses($texts, $hashOf, $vectors);

        if ([] !== $misses) {
            if (!$this->ollamaEnabled) {
                // brak wektora to błąd, nigdy cichy powrót do innej metody
                throw new \RuntimeException('Embedding cache miss …');
            }

            $fresh = $this->inner->vectorize($misses);      // jedyne wywołanie modelu
            $this->store->putMany($fresh, $this->modelId);  // zapis przed użyciem
            $vectors += $fresh;
        }

        return array_map(static fn (string $h) => $vectors[$h], $hashOf);
    }

    public static function keyFor(string $text): string
    {
        return hash('sha256', trim($text));   // klucz z tekstu dla modelu
    }
}
```

Klucz w bazie to para `(skrót tekstu, nazwa modelu)`, a wiersz zawiera wektor zapisany jako `jsonb`. Nie ma terminu ważności, ponieważ wartość się nie zmienia.

TRZY MOŻLIWE SCENARIUSZE

TEKST JUŻ ZNANY
`dowolny tryb`
Gdy ten sam tekst pojawia się ponownie, program odczytuje liczby z bazy bez uruchamiania modelu.

NOWY TEKST, MODEL DOSTĘPNY
`wywołania modelu włączone`
Model wykonuje obliczenia lokalnie, po trzy teksty naraz. Wynik od razu trafia do bazy, więc przy ponownym przetwarzaniu tego tekstu nie trzeba już uruchamiać modelu.

NOWY TEKST, MODEL WYŁĄCZONY
`wywołania modelu wyłączone`
Program zatrzymuje się, wskazuje tekst, dla którego brakuje wyniku, oraz podaje polecenie pozwalające go przygotować. Nie używa w zamian słabszej metody.

OD STRONY TECHNICZNEJ

Tabela `embedding_cache`, klucz złożony `(sha256(trim(text)), model_id)`, wektor jako `jsonb`, bez TTL. Przy zapisie używane jest `ON CONFLICT DO NOTHING`, więc równoległe procesy robocze nie wchodzą w konflikt. Nazwa modelu jest częścią klucza: po jej zmianie program nie odczytuje już wcześniejszych wierszy dla tego samego tekstu. Wektor jest liczony od nowa. Wektory poprzedniego modelu zostają w tabeli, ale nie są już odczytywane, więc poprawność nie zależy od tego, czy ktoś pamięta o wyczyszczeniu pamięci podręcznej. Ustawienie: `OLLAMA_ENABLED`. Inicjalizacja pamięci podręcznej: `bin/console app:embed:warm --days=1`.

PRÓBKA

JAK WYGLĄDA ZAPISANY WEKTOR

Nagłówek o rezygnacji Macrona jest fikcyjny i powstał na potrzeby scenariusza testowego. Wektor zawiera wartości rzeczywiście obliczone przez działający model dla tego nagłówka i zapisane w danych testowych repozytorium. System produkcyjny zapisuje wektory w tym samym formacie. Jeden nagłówek na wejściu, jeden wiersz 768 liczb na wyjściu.

JEDEN NAGŁÓWEK, JEDEN WIERSZ

W tym przypadku pole treści było puste, więc model otrzymał sam nagłówek. Klucz składa się z jego skrótu i nazwy modelu.

CO PRZECZYTAŁ MODEL	Macron resignation abdication power president france Macron resignation
SKRÓT TEKSTU	7319e9beaf703b2351870019ae5446cf94cefe520f5f997f6326b3a357d62169
MODEL	nomic-embed-text-v2-moe
WYNIK	768 liczb · długość 1,00 · około 9,3 kB w bazie

OD STRONY TECHNICZNEJ

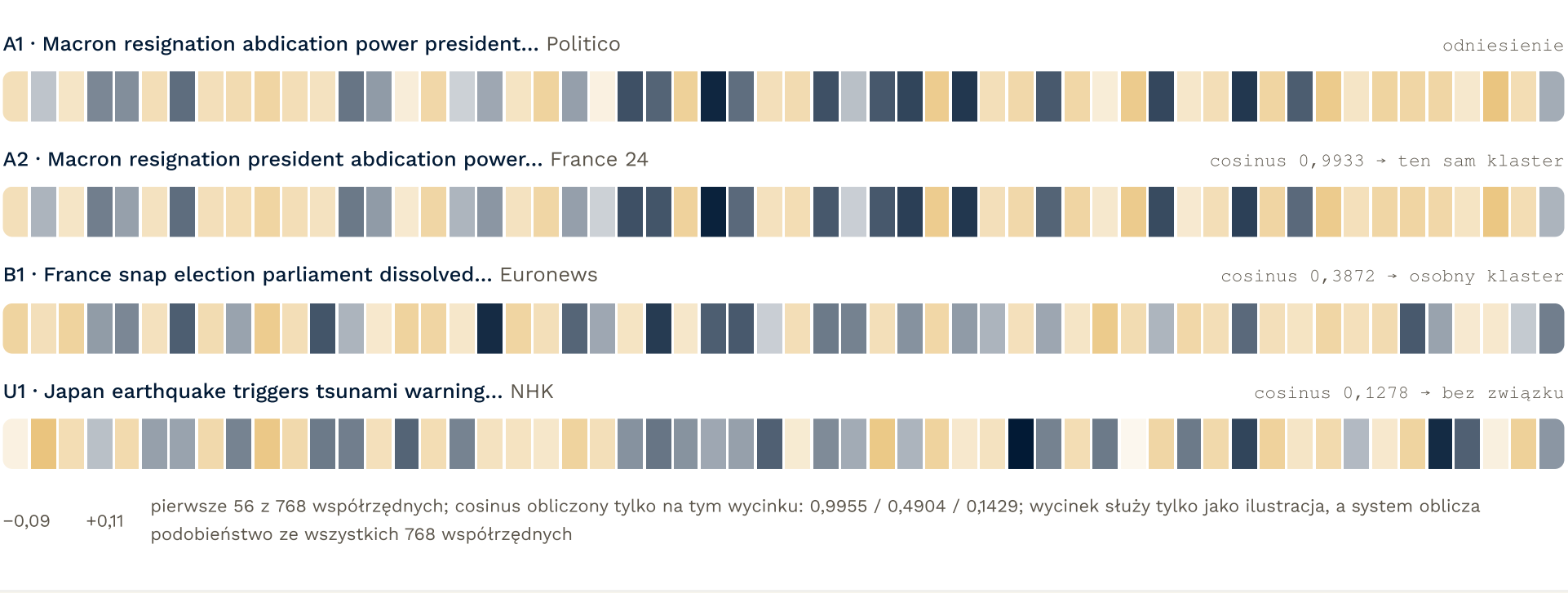
WIERSZ NA DYSKU

Tabela ma trzy kolumny: `VARCHAR(64)`, `VARCHAR(255)` i `JSONB`, a także złożony klucz główny. Innych indeksów nie potrzeba: wyszukiwanie odbywa się po dokładnym kluczu, nigdy po podobieństwie.

```
-- SELECT * FROM embedding_cache LIMIT 1;
text_hash 7319e9beaf703b2351870019ae5446cf…
model_id nomic-embed-text-v2-moe
vector [0.021463, -0.003320, 0.013590, -0.022463,
… 763 kolejne wartości …
-0.026217, 0.054305]
```

PIERWSZE 56 WSPÓŁRZĘDNYCH CZTERECH WEKTORÓW

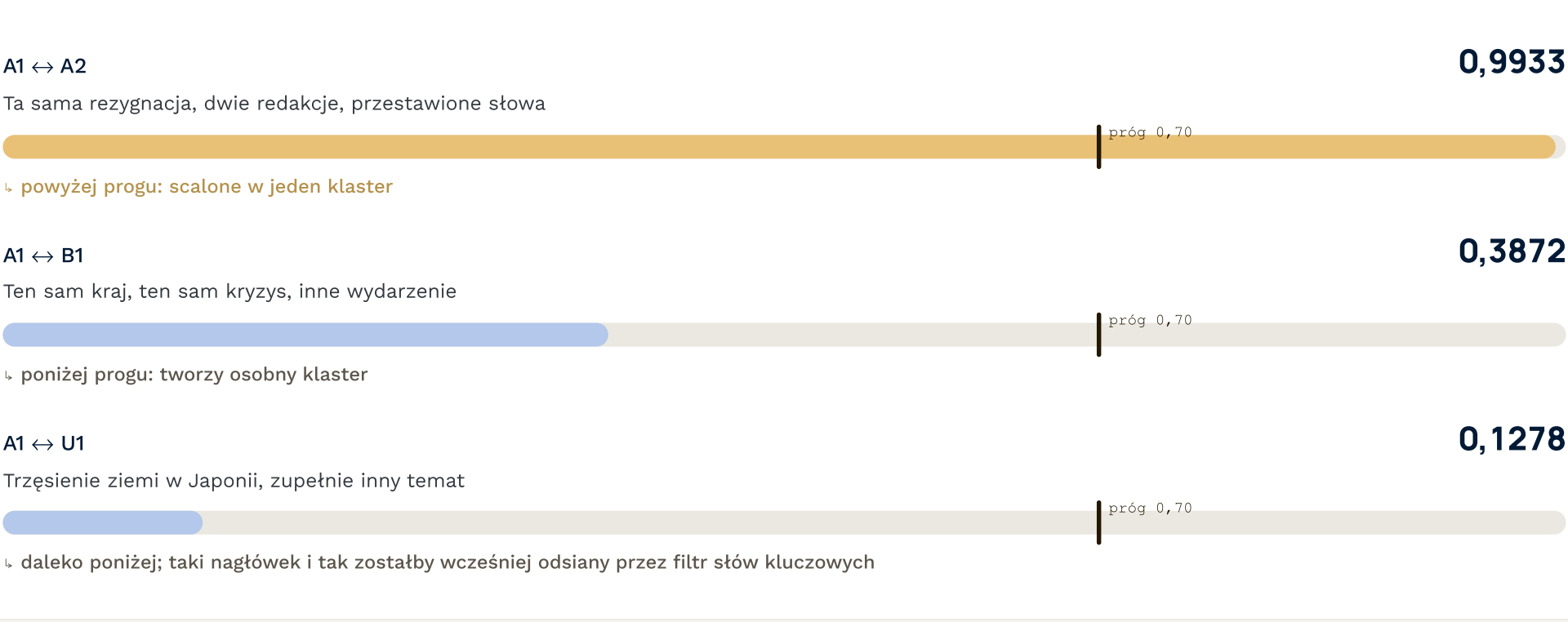
Każdy kwadrat odpowiada jednej współrzędnej. Barwa piaskowa oznacza wartość dodatnią, a granatowa ujemną. Im intensywniejsza barwa, tym większa wartość bezwzględna. Dwa pierwsze nagłówki mówią o rezygnacji Macrona innymi słowami, a mimo to ich wzory są niemal identyczne. Trzeci również dotyczy francuskiej polityki, ale opisuje przedterminowe wybory, więc daje wyraźnie odmienny wzór. Czwarty, o trzęsieniu ziemi w Japonii, nie przypomina żadnego z poprzednich.



pierwsze 56 z 768 współrzędnych; cosinus obliczony tylko na tym wycinku: 0,9955 / 0,4904 / 0,1429; wycinek służy tylko jako ilustracja, a system oblicza podobieństwo ze wszystkich 768 współrzędnych

GDZIE WYPADA PRÓG

Poniższe wartości pokazują podobieństwo każdego nagłówka do A1. Wynik dla tej samej historii jest wyraźnie wyższy od pozostałych. W tej próbce próg 0,70 leży między nimi.



WYBORY

CZTERY DECYZJE PROJEKTOWE

01 DWA SKRÓTY ZAMIAST JEDNEGO

Każdy artykuł ma skrót treści służący do rozpoznawania duplikatów. Oba skróty powstają z nagłówka i treści, ale skrót treści nie może bezpiecznie pełnić funkcji klucza pamięci podręcznej. Po zmianie sposobu składania tekstu przekazywanego modelowi skrót służący do wykrywania duplikatów pozostałby bez zmian, choć model dostałby inny tekst. Pamięć podręczna zwracałaby wtedy liczby obliczone dla innej treści, bez żadnego ostrzeżenia. Osobny skrót wymaga zaledwie kilku linii kodu i zapobiega takiej sytuacji.

02 ZATRZYMANIE ZAMIAST PRZEJŚCIA NA INNĄ METODĘ

Gdyby przy niedostępnym modelu program przechodził na porównywanie słów, nadal zwracałby wiarygodnie wyglądające grupy. Byłyby jednak ułożone inaczej, a nic w wyniku nie wskazywałoby, za pomocą której z dwóch metod powstały. Grupy artykułów służą tutaj do tworzenia sygnałów, które mogą później wspierać decyzję transakcyjną podejmowaną poza aplikacją. Błędne pogrupowanie może więc prowadzić do błędnego sygnału. Bezpieczniej zatrzymać program z czytelnym komunikatem, niż przeoczyć pogorszenie jakości wyniku.

03 JEDNA METODA GRUPOWANIA WE WSZYSTKICH TRYBACH

Ustawienie w konfiguracji określa wyłącznie, czy program może wywołać model; nie zmienia metody grupowania. Testy i tryb pracy bez dostępu do sieci korzystają z wyników zapisanych razem z kodem. Przetwarzają je tak samo jak system produkcyjny.

OD STRONY TECHNICZNEJ

`OLLAMA_ENABLED` określa, czy przy braku wyniku w pamięci podręcznej wolno wywołać model; nie wybiera strategii grupowania. Dane testowe z pliku `tests/fixtures/embeddings/nomic-embed-text-v2-moe.json` są wczytywane do `embedding_cache` przed każdym scenariuszem. We wszystkich trzech trybach wywoływana jest klasa `CachedTextVectorizer`.

04 ZWYKŁY JSON ZAMIAST BAZY WEKTOROWEJ

Wyszukiwanie odbywa się po dokładnym kluczu, a porównania nie wykraczają poza jedną niewielką porcję artykułów. Wyspecjalizowany indeks podobieństwa nie przyniósłby tu korzyści. Wymusiłby za to przechowywanie wektorów o jednym rozmiarze, tymczasem magazyn ma obsługiwać wyniki różnych modeli. Sytuacja zmieni się, gdy program zacznie porównywać nowe wydarzenia z trwałym archiwum.

SZERZEJ

DO CZEGO JESZCZE UŻYWA SIĘ EMBEDDINGÓW

W każdym z wymienionych zastosowań mechanizm jest ten sam. Model umieszcza treści w jednej przestrzeni, a system porównuje ich wektory za pomocą prostej operacji matematycznej.

`tak działa Proximtype`
GRUPOWANIE I WYKRYWANIE TEMATÓW
Teksty bez przypisanych kategorii układają się w skupiska, a każde z nich odpowiada jednemu tematowi, także takiemu, którego nikt wcześniej nie przewidział.

`tak działa Proximtype`
DOPASOWANIE MIĘDZY JĘZYKAMI
Doniesienia po francusku, angielsku i polsku mogą trafić do jednej grupy bez wcześniejszego tłumaczenia.

`powszechne`
WYSZUKIWANIE SEMANTYCZNE
Model umieszcza pytanie i dokumenty w tej samej przestrzeni, więc w odpowiedzi na pytanie „jak oddać zepsuty towar” wyszukiwarka podaje „procedurę reklamacyjną”, choć te dwa sformułowania nie mają wspólnego słowa.

`powszechne`
DOSTARCZANIE MODELOWI KONTEKSTU (RAG)
Zanim pytanie trafi do dużego modelu, system dołącza do niego fragmenty własnych dokumentów, wybrane na podstawie podobieństwa do pytania. Model może wtedy oprzeć odpowiedź na materiale organizacji, zamiast polegać wyłącznie na własnej pamięci.

`powszechne`
DUPLIKATY, KTÓRE NIE SĄ IDENTYCZNE
Może to być przeklejone ogłoszenie ze zmienionym numerem telefonu albo ta sama sprawa zgłoszona po raz drugi innymi słowami. Skrót treści uzna je za różne, ale embedding może pomóc je ze sobą powiązać.

`powszechne`
KLASYFIKACJA BEZ TRENOWANIA
Każdą kategorię opisuje jedno zdanie, a tekst trafia do tej, której opis leży najbliżej. Dodanie kategorii wymaga kolejnego zdania, a nie ponownego zbierania danych i trenowania klasyfikatora.

`powszechne`
REKOMENDACJE I „PODOBNE DO TEGO”
System znajduje najbliższych sąsiadów w przestrzeni i na tej podstawie podpowiada podobne artykuły, produkty, oferty pracy czy zgłoszenia. Nie wymaga to ręcznego utrzymywania taksonomii, czyli uporządkowanego systemu kategorii.

`powszechne`
KIEROWANIE ZGŁOSZEŃ
Nowe zgłoszenie trafia do zespołu, którego dotychczasowe sprawy leżą najbliżej. System opiera się na historii tych spraw, nie na zestawie reguł.

`powszechne`
WYCHWYTYWANIE ODSTĘPSTW
Tekst odległy od wszystkich dotychczasowych materiałów może być przypadkowym szumem albo pierwszą oznaką czegoś nowego. W obu przypadkach warto mu się przyjrzeć.

✕ CZEGO EMBEDDING NIE ZROBI

Nie ustala, czy dwa teksty są identyczne. Aby sprawdzić, czy ktoś przysłał dokładnie ten sam tekst, wystarczy skrót treści z kroku 01. Sam wynik nie jest też wyjaśnieniem: pokazuje, że dwa teksty są blisko siebie, ale nie mówi, dlaczego. Model korzysta tylko z zależności, których nauczył się podczas trenowania. Gdy artykuł dotyczy wydarzenia z zeszłego tygodnia, wektor opisuje tylko przekazany tekst.

Port `MarketSignals/Domain/WorldEvent/TextVectorizerInterface` **Adaptery** `CachedTextVectorizer → OllamaTextVectorizer`
Magazyn `embedding_cache (text_hash, model_id) → jsonb` **Model** `nomic-embed-text-v2-moe · 768 wymiarów`
Inicjalizacja pamięci podręcznej `bin/console app:embed:warm --days=1`